

138. 随机链表的复制

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium (69.41%)	1598	0	-	-	0

- Tags
- Companies

给你一个长度为 n 的链表，每个节点包含一个额外增加的随机指针 `random`，该指针可以指向链表中的任何节点或空节点。

构造这个链表的 深拷贝。深拷贝应该正好由 n 个 全新 节点组成，其中每个新节点的值都设为其对应的原节点的值。新节点的 `next` 指针和 `random` 指针也都应指向复制链表中的新节点，并使原链表和复制链表中的这些指针能够表示相同的链表状态。复制链表中的指针都不应指向原链表中的节点。

例如，如果原链表中有 X 和 Y 两个节点，其中 $X.random \rightarrow Y$ 。那么在复制链表中对应的两个节点 x 和 y ，同样有 $x.random \rightarrow y$ 。

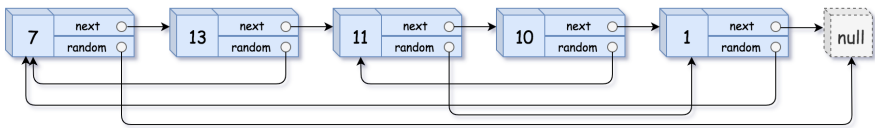
返回复制链表的头节点。

用一个由 n 个节点组成的链表来表示输入/输出中的链表。每个节点用一个 `[val, random_index]` 表示：

- `val`: 一个表示 `Node.val` 的整数。
- `random_index`: 随机指针指向的节点索引（范围从 0 到 $n-1$ ）；如果不指向任何节点，则为 `null`。

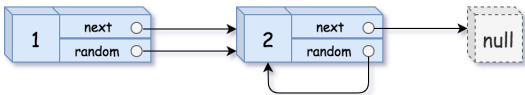
你的代码 只 接受原链表的头节点 `head` 作为传入参数。

示例 1:



输入: `head = [[7,null],[13,0],[11,4],[10,2],[1,0]]`
输出: `[[7,null],[13,0],[11,4],[10,2],[1,0]]`

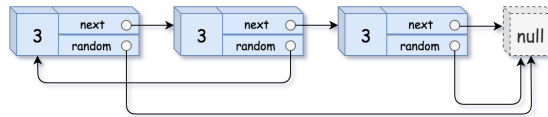
示例 2:



输入: `head = [[1,1],[2,1]]`

输出: `[[1,1],[2,1]]`

示例 3:



输入: `head = [[3,null],[3,0],[3,null]]`

输出: `[[3,null],[3,0],[3,null]]`

提示:

- $0 \leq n \leq 1000$
- $104 \leq \text{Node.val} \leq 104$
- `Node.random` 为 `null` 或指向链表中的节点。

Discussion | Solution

2. 方法

LeetCode 138: 随机链表的复制（详细解析）

1. 问题理解

目标: 创建一个给定链表的深层副本 (**deep copy**)。这个链表比较特殊, 每个节点除了有 `val` 和 `next` 指针外, 还有一个 `random` 指针, 它可以指向链表中的任意节点或者 `null`。

深层副本定义:

- 创建一个全新的链表, 包含全新的节点。
- 新链表中每个节点的值 (`val`) 与原链表中对应节点的值相同。
- 新链表中节点的 `next` 指针指向新链表中的下一个节点, 结构与原链表相同。
- 新链表中节点的 `random` 指针指向新链表中的对应节点, 结构与原链表相同。
- 修改副本链表不应影响原始链表。

输入: 特殊链表的头节点 `head`。

输出: 深层副本链表的头节点。

核心挑战: 如何在创建新节点的同时, 正确地复制 `random` 指针的指向关系? 直接复制 `random` 指针会使得新旧链表共享节点, 不是深层副本。我们需要让新节点的 `random` 指针指向新链表中对应的节点。

2. 思路分析

思路一：使用哈希映射 (**HashMap**)

想法：利用哈希映射存储原始节点到新创建的副本节点之间的映射关系。这样，在设置 **next** 和 **random** 指针时，可以通过原始节点快速找到其对应的副本节点。

步骤：

1. 第一次遍历 (创建节点和映射):
 - 遍历原始链表。
 - 对于每个原始节点 `originalNode`，创建一个新的副本节点 `copyNode`，其 `val` 与 `originalNode.val` 相同。
 - 将这个映射关系 (`originalNode -> copyNode`) 存入哈希映射 `map` 中。
2. 第二次遍历 (设置指针):
 - 再次遍历原始链表。
 - 对于每个原始节点 `originalNode`:
 - 从 `map` 中获取其对应的副本节点 `copyNode = map.get(originalNode)`。
 - 设置 `copyNode` 的 `next` 指针: `copyNode.next = map.get(originalNode.next)` (注意处理 `originalNode.next` 为 `null` 的情况)。
 - 设置 `copyNode` 的 `random` 指针: `copyNode.random = map.get(originalNode.random)` (注意处理 `originalNode.random` 为 `null` 的情况)。
3. 返回 `map.get(head)` 作为副本链表的头节点。

复杂度：

- 时间复杂度: $O(N)$ ，需要两次遍历链表。哈希映射操作平均为 $O(1)$ 。
- 空间复杂度: $O(N)$ ，需要哈希映射存储 N 个节点的映射关系。

思路二：原地修改 + 拆分 (**$O(1)$** 空间)

想法：通过在原始链表中原地插入副本节点，建立一种临时的、交错的结构，使得可以不通过额外的数据结构就能找到原始节点与其副本节点的对应关系，以及 **random** 指针目标节点与其副本节点的对应关系。

步骤：

1. 第一次遍历 (复制并插入节点):
 - 遍历原始链表。
 - 对于每个原始节点 `node`:
 - 创建一个副本节点 `copyNode`，`copyNode.val = node.val`。
 - 将 `copyNode` 插入到 `node` 之后:
 - * `copyNode.next = node.next`
 - * `node.next = copyNode`
 - 完成后，链表结构变为: `A -> A' -> B -> B' -> C -> C' -> ...`
2. 第二次遍历 (设置 **random** 指针):
 - 再次遍历这个交错的链表 (从 `head` 开始)。
 - 对于每个原始节点 `node`:

- 其副本节点是 `copyNode = node.next`。
 - 如果 `node.random` 不为 `null`，那么 `node.random` 指向的原始目标节点的副本就是 `node.random.next`。
 - 设置 `copyNode.random = (node.random != null) ? node.random.next : null`。
3. 第三次遍历 (拆分链表):
- 将交错的链表拆分成原始链表和副本链表。
 - 需要维护原始链表的当前指针 `originalCurr` 和副本链表的当前指针 `copyCurr`。
 - 使用一个哑节点 `dummyCopyHead` 来方便构建副本链表。
 - 遍历交错链表:
 - `originalCurr = head, copyCurr = dummyCopyHead`
 - 对于每个原始节点 `originalCurr`:
 - * 获取副本节点 `copyNode = originalCurr.next`。
 - * 将 `copyNode` 连接到副本链表: `copyCurr.next = copyNode;`
`copyCurr = copyNode;`
 - * 恢复原始链表的 `next` 指针: `originalCurr.next = copyNode.next;`
 - * 移动原始链表指针: `originalCurr = originalCurr.next;`
 - 返回 `dummyCopyHead.next`。

复杂度:

- 时间复杂度: $O(N)$ ，需要三次遍历链表。
- 空间复杂度: $O(1)$ ，只使用了常数个额外指针（不考虑创建副本链表本身所需的空间）。

3. 代码实现 ($O(1)$ 空间法)

```

/*
// Definition for a Node.
class Node {
    int val;
    Node next;
    Node random;

    public Node(int val) {
        this.val = val;
        this.next = null;
        this.random = null;
    }
}
*/

class Solution {
    /**
     * 复制带随机指针的链表 -  $O(1)$  空间复杂度方法

```

```

* @param head 原始链表头节点
* @return 副本链表头节点
*/
public Node copyRandomList(Node head) {
    if (head == null) {
        return null;
    }

    // 1. 复制节点并插入到原节点后面 A -> A' -> B -> B' -> ...
    Node current = head;
    while (current != null) {
        Node copyNode = new Node(current.val);
        copyNode.next = current.next;
        current.next = copyNode;
        current = copyNode.next; // 移动到原始链表的下一个节点
    }

    // 2. 设置副本节点的 random 指针
    current = head;
    while (current != null) {
        Node copyNode = current.next;
        if (current.random != null) {
            // current.random 是原始 random 指针指向的节点
            // current.random.next 是该原始节点对应的副本节点
            copyNode.random = current.random.next;
        }
        current = copyNode.next; // 移动到原始链表的下一个节点
    }

    // 3. 拆分原始链表和副本链表
    Node dummyCopyHead = new Node(0); // 副本链表的哑头节点
    Node copyTail = dummyCopyHead;    // 副本链表的尾指针
    current = head;                    // 原始链表的遍历指针

    while (current != null) {
        Node copyNode = current.next; // 获取副本节点
        Node originalNext = copyNode.next; // 获取原始链表的下一个节点

        // 连接副本节点
        copyTail.next = copyNode;
        copyTail = copyNode; // 移动副本链表尾指针, copyNode.next 会在下一步被修改, 所以先移

        // 恢复原始链表
        current.next = originalNext;

        // 移动原始链表指针
    }
}

```

```

        current = originalNext;
    }

    return dummyCopyHead.next;
}
}

```

4. 代码解析 ($O(1)$ 空间法)

- 步骤 1 (复制与插入): 遍历原始链表, 对每个节点 `current`, 创建一个值相同的 `copyNode`, 然后将 `copyNode` 插入到 `current` 和 `current.next` 之间。`current` 指针每次跳两步 (`current = copyNode.next`) 以继续遍历原始节点。
- 步骤 2 (设置 `random`): 再次遍历交错链表。对于每个原始节点 `current`, 其副本 `copyNode` 就是 `current.next`。`current.random` 指向原始目标节点, 那么 `current.random.next` 就是该目标节点对应的副本节点。据此设置 `copyNode.random`。`current` 指针同样每次跳两步。
- 步骤 3 (拆分): 这是最需要小心指针操作的一步。
 - 使用 `dummyCopyHead` 和 `copyTail` 来构建副本链表。
 - `current` 遍历原始节点。
 - 在循环中:
 - * 先保存副本节点 `copyNode = current.next` 和原始链表的下一个节点 `originalNext = copyNode.next`。
 - * 将 `copyNode` 连接到副本链表的尾部 (`copyTail.next = copyNode`), 并更新 `copyTail`。
 - * 恢复 `current` 的 `next` 指针, 指向正确的下一个原始节点 (`current.next = originalNext`)。
 - * 将 `current` 移动到下一个原始节点 (`current = originalNext`)。

5. 如何在面试中讲解

1. 理解问题和挑战:
 - “我们需要创建一个链表的深层副本, 这个链表有 `next` 和 `random` 指针。主要挑战在于如何正确复制 `random` 指针, 让它指向副本链表中的对应节点。”
2. 提出哈希映射方法 (作为对比或初步方案):
 - “一种直接的方法是使用哈希映射。第一次遍历创建所有副本节点并将 `< 原始节点, 副本节点 >` 存入映射。第二次遍历根据映射关系设置副本节点的 `next` 和 `random` 指针。”
 - “这种方法时间复杂度 $O(N)$, 空间复杂度 $O(N)$ 。”
3. 引入 $O(1)$ 空间方法 (重点讲解):
 - “如果要求 $O(1)$ 的额外空间复杂度, 我们可以采用一种原地修改和拆分的方法, 分三步进行。”
4. 详细讲解 $O(1)$ 空间法的步骤:
 - 第一步: 复制与交织

- “遍历原始链表，对于每个节点 **A**，创建一个副本 **A'**，并将 **A'** 插入到 **A** 之后，形成 **A -> A' -> B -> B' -> ...** 的结构。”
 - “这样，对于任何原始节点 **X**，它的副本 **X'** 就在 **X.next**。”
 - 第二步：设置 **random** 指针
 - “再次遍历这个交织链表。对于每个原始节点 **A**，它的副本是 **A' = A.next**。”
 - “如果 **A.random** 指向原始节点 **R**，那么 **R** 的副本 **R'** 就是 **R.next**。”
 - “所以，我们可以设置 **A'.random = A.random.next** (需要处理 **A.random** 为 **null** 的情况)。”
 - 第三步：拆分链表
 - “最后，我们需要将这个交织链表拆分成原始链表和副本链表。”
 - “遍历交织链表，同时维护原始链表和副本链表的尾指针。”
 - “对于每个原始节点 **A** 和其副本 **A'**：将 **A'** 从交织链表中取出并连接到副本链表的尾部；同时恢复 **A** 的 **next** 指针，指向下一个原始节点 **B**。”
 - “使用一个哑节点可以方便地构建副本链表。”
5. 分析复杂度：
- “这个方法需要三次遍历，所以时间复杂度仍然是 $O(N)$ 。”
 - “但它只使用了常数个额外指针，所以空间复杂度是 $O(1)$ (不计入副本链表本身的空间)。”
6. 总结：
- “ $O(1)$ 空间的方法虽然步骤多一些，指针操作更复杂，但空间效率更高。”

在讲解 $O(1)$ 方法时，如果能画图辅助说明交织和拆分的过程，效果会更好。